

VISTAS

En el fascinante mundo de las bases de datos y la gestión de datos, existen herramientas y conceptos que nos permiten optimizar nuestras consultas, simplificar la manipulación de datos y mejorar el rendimiento en general.

*En este contexto, las **vistas** en SQL se presentan como una **función poderosa y versátil** que nos ofrece un enfoque eficiente y organizado para trabajar con datos en entornos SQL.*





VISTAS

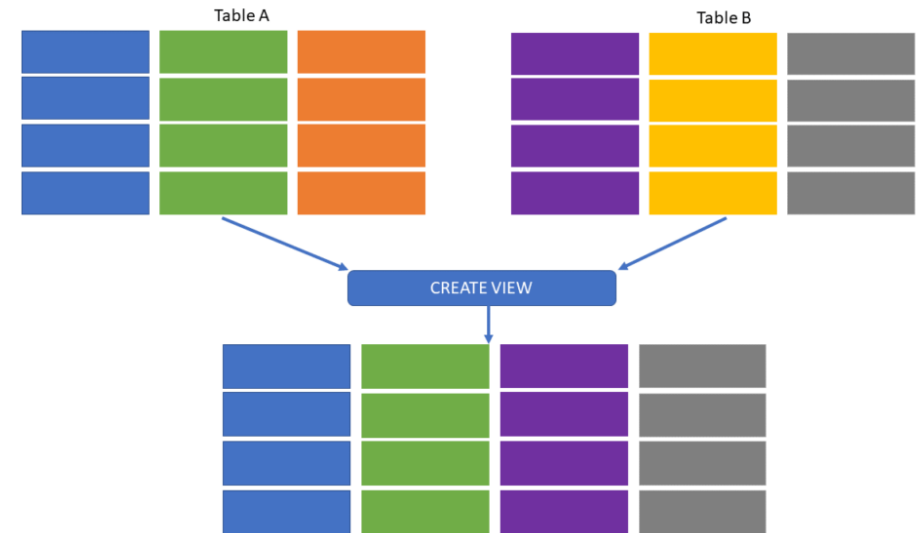
Las vistas en SQL son objetos virtuales que nos permiten acceder y manipular datos almacenados en una o varias tablas como si fueran tablas individuales.

En esencia, una vista es una consulta predefinida que se guarda en la base de datos y se comporta como una tabla virtual.

Se utilizan para simplificar consultas completas y puede contener proyecciones, filtros, uniones y otras operaciones SQL que nos permiten seleccionar y presentar los datos de una manera específica.

Una **tabla subyacente** es una tabla en la base de datos que sirve como **fFuente de datos** para una **vista**

(p.ej. EMPLEADOS)



```
CREATE VIEW VistaVentas AS
SELECT Nombre, Apellido, Salario
FROM Empleados
WHERE Departamento = 'Ventas';

SELECT * FROM VistaVentas;
```



VISTAS

```
CREATE VIEW identificador AS  
SELECT *  
FROM tabla  
WHERE condición;
```

- 1) *Simplificación de consultas*
- 2) *Organización de datos*
- 3) *Seguridad y control de acceso*
- 4) *Rendimiento mejorado*
- 5) *Abstracción de la estructura de datos*

-- Vistas simples:

```
CREATE VIEW VistaEmpleados AS  
SELECT Nombre, Apellido, Salario  
FROM Empleados  
WHERE Departamento = 'Ventas';
```

-- Vistas complejas:

```
CREATE VIEW VistaVentas AS  
SELECT e.Nombre, e.Apellido, SUM(v.Cantidad) AS TotalVentas  
FROM Empleados e  
JOIN Ventas v ON e.IdEmpleado = v.IdEmpleado  
GROUP BY e.Nombre, e.Apellido;
```

-- Vistas indexadas:

```
CREATE VIEW VistaSalarios  
WITH SCHEMABINDING AS  
SELECT IdEmpleado, Salario  
FROM dbo.Empleados;  
GO  
CREATE UNIQUE CLUSTERED INDEX idx_salario  
ON VistaSalarios (Salario);
```

-- Vistas con columnas calculadas:

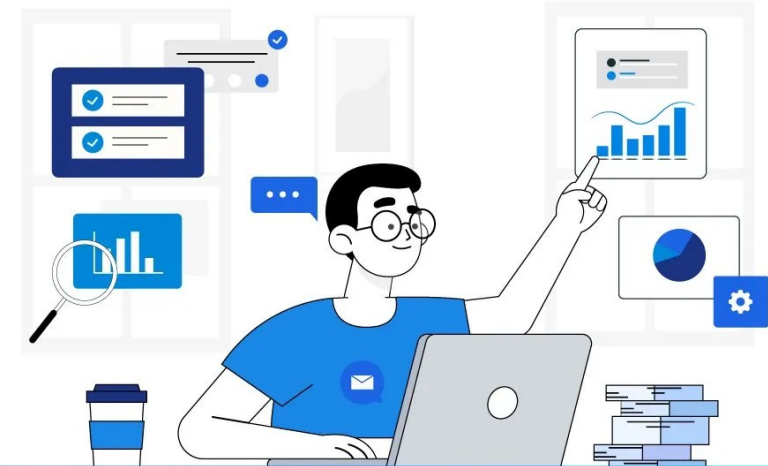
```
CREATE VIEW VistaSalarioAnual AS  
SELECT Nombre, Apellido, Salario, (Salario * 12) AS  
SalarioAnual  
FROM Empleados;
```



VISTAS

Requisitos para que una vista sea actualizable:

1. **Debe estar basada en una sola tabla:** Las vistas que combinan varias tablas o usan JOINS generalmente no son actualizables.
2. **No debe contener funciones agregadas** (SUM(), COUNT(), etc.).
3. **No debe usar DISTINCT, GROUP BY, o HAVING.**
4. **No debe contener subconsultas.**
5. **Todas las columnas necesarias** para realizar la actualización (por ejemplo, las claves primarias) deben estar incluidas en la vista.



-- Ejemplo de una vista actualizable:

```
CREATE VIEW VistaSimple AS  
SELECT IdEmpleado, Nombre, Apellido, Salario  
FROM Empleados;
```

--Insertar en la vista:

```
INSERT INTO VistaSimple (IdEmpleado, Nombre, Apellido,  
Salario)  
VALUES (6, 'Carlos', 'López', 4500);
```




VISTAS

1. **Identificar y Analizar el Problema:** usar herramientas como **Execution Plan** – Ctrl + M : *Include Actual Execution Plan*

2. **Revisar y simplificar la consulta:**
Evitar subconsultas innecesarias.
Evitar SELECT *

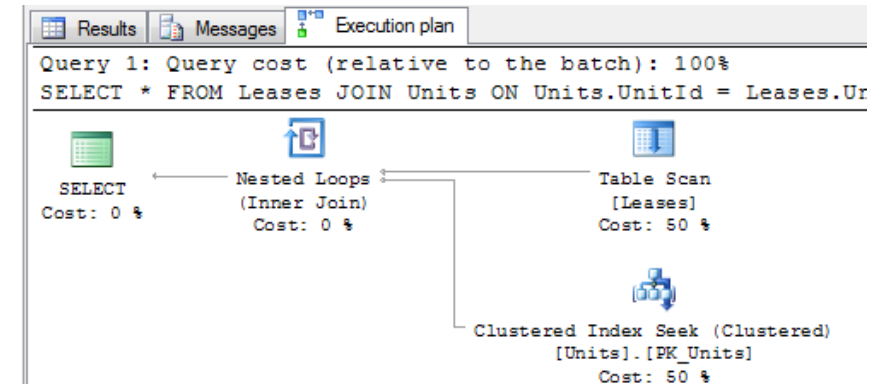
3. **Usar índices apropiados:**
Columnas filtradas
Índices compuestos

4. **Optimizar Joins y Subconsultas**

5. **Optimizar Funciones Agregadas y GROUP BY**

6. **Limitar datos recuperados:** usar LIMIT o TOP.

7. **Particionamiento de Tablas**



VISTAS

-- Vista para mostrar información integrada del empleado

CREATE VIEW RecursosHumanos.vw_EmpleadosConDatos AS

SELECT

E.IdEmpleado,
E.Nombre AS NombreEmpleado,
E.Salario,
D.NombreDepartamento,
C.NombreCategoria,
H.NombreHijo,
H.FechaNacimiento,
ES.Descripcion AS TituloEstudio

FROM

RecursosHumanos.Empleados E

LEFT JOIN RecursosHumanos.Departamento D ON E.IdDepartamento = D.IdDepartamento

LEFT JOIN RecursosHumanos.Categoria C ON E.IdCategoria = C.IdCategoria

LEFT JOIN RecursosHumanos.Hijos H ON E.IdEmpleado = H.IdEmpleado

LEFT JOIN RecursosHumanos.Estudios ES ON E.IdEmpleado = ES.IdEmpleado;

Ejecutar Vista

SELECT * FROM RecursosHumanos.vw_EmpleadosConDatos

¿Cómo mejorar la seguridad y la eficiencia?

Se refiere a cómo las bases de datos permiten crear capas intermedias **abstracciones** que ocultan o simplifican la interacción con los datos subyacentes, haciendo más fácil gestionar el acceso a la información y mejorar el rendimiento.

```
CREATE VIEWSELECT View_Empleados AS  
SELECT Nombre, Cargo, Departamento FROM Empleados;
```

Control de Acceso

Manejo de Permisos

SQL - *Structured Query Language*



Objetivo: Comprender la importancia de la seguridad en bases de datos, aprender a administrar perfiles de usuario y permisos mediante Grant y Revoke, y manejar anidamientos y operadores de conjunto como ALL y ANY.

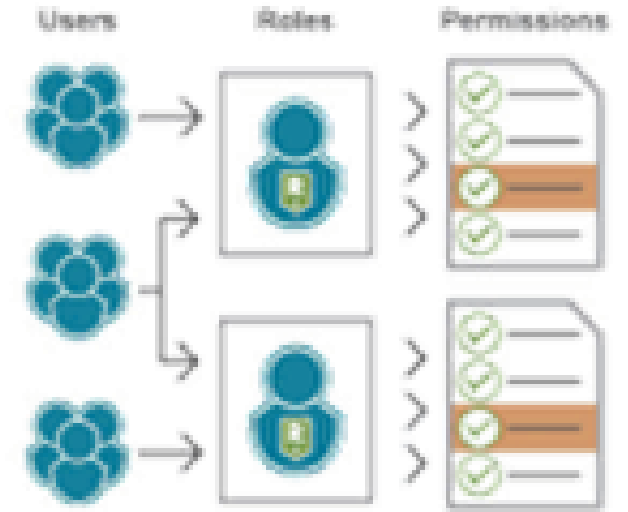
...descubriendo el poder de los datos

2025

LENGUAJE DE CONTROL DE DATOS

Niveles de seguridad:

- **Autenticación:** Verificación de identidad (usuarios y contraseñas).
 - Autenticación de Windows: Utiliza las credenciales del sistema operativo para validar al usuario.
 - Autenticación de SQL Server: Requiere un login y una contraseña específicos del servidor SQL.
- **Autorización:** Control del acceso a los recursos mediante permisos.



LENGUAJE DE CONTROL DE DATOS

Crear un login:

Este login es para autenticación mediante usuario y contraseña (Autenticación de SQL Server).

```
CREATE LOGIN login_ejemplo WITH PASSWORD = 'ContraseñaSegura2024';
```

Crear un usuario asociado al login en una base de datos:

Una vez que el login está creado, se crea el usuario en una base de datos específica.

```
USE MiBaseDeDatos;  
CREATE USER usuario_ejemplo FOR LOGIN login_ejemplo;
```



LENGUAJE DE CONTROL DE DATOS

Trabajo con Esquemas

Los esquemas permiten organizar los objetos dentro de una base de datos. Puedes asignar un usuario a un esquema específico para controlar su acceso a los objetos dentro de ese esquema.

```
CREATE SCHEMA esquema_ejemplo;
```

Asignar un usuario a un esquema:

Cuando creas un usuario, puedes asociarlo a un esquema para que tenga acceso a los objetos que pertenecen a ese esquema.

```
ALTER USER usuario_ejemplo WITH DEFAULT_SCHEMA = esquema_ejemplo;
```



LENGUAJE DE CONTROL DE DATOS

-- Dar permisos de SELECT en una tabla:

```
GRANT SELECT ON esquema_ejemplo.tabla_ejemplo TO usuario_ejemplo;
```

-- Dar permisos de INSERT y UPDATE en una tabla:

```
GRANT INSERT, UPDATE ON esquema_ejemplo.tabla_ejemplo TO usuario_ejemplo;
```

-- Dar permisos a un procedimiento almacenado: Si el usuario necesita ejecutar un procedimiento almacenado:

```
GRANT EXECUTE ON SCHEMA::esquema_ejemplo TO usuario_ejemplo;
```



LENGUAJE DE CONTROL DE DATOS

-- Revocar permiso de SELECT en una tabla:

```
REVOKE SELECT ON esquema_ejemplo.tabla_ejemplo FROM usuario_ejemplo;
```

-- Revocar permisos de INSERT y UPDATE:

```
REVOKE INSERT, UPDATE ON esquema_ejemplo.tabla_ejemplo FROM usuario_ejemplo;
```

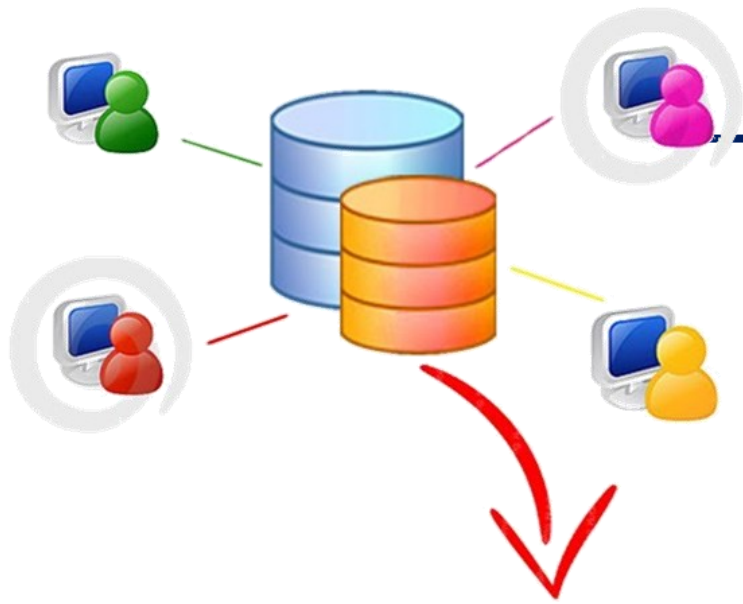


LENGUAJE DE CONTROL DE DATOS

-- Denegar permisos de DELETE en una tabla:

```
DENY DELETE ON esquema_ejemplo.tabla_ejemplo TO usuario_ejemplo;
```





ROLES EN SQL SERVER

Los **roles de base de datos** controlan qué puede hacer un usuario dentro de una base de datos específica. SQL Server incluye varios roles predefinidos que otorgan diferentes niveles de acceso a los usuarios sobre objetos y recursos en la base de datos.

- **db_owner**: Tiene control total sobre la base de datos y puede realizar cualquier tarea.
- **db_datareader**: Permite leer todos los datos de las tablas y vistas dentro de la base de datos.
- **db_datawriter**: Permite insertar, actualizar y eliminar datos en todas las tablas y vistas.
- **db_securityadmin**: Puede modificar roles y permisos en la base de datos, SIN ACCESO A DATOS.
- **db_backupoperator**: Puede realizar backups de la base de datos, SIN ACCESO A DATOS.

```
USE MiBaseDeDatos;  
ALTER ROLE db_owner ADD MEMBER nombre_usuario;
```

CONSULTAS ANIDADAS



Subconsultas

Una **consulta anidada**, también conocida como **subconsulta**, es una consulta SQL que se encuentra dentro de otra consulta.

Se utilizan para devolver datos que serán utilizados por la consulta principal (o externa) para realizar una acción adicional, como comparaciones o filtrado de resultados.

Devuelve un conjunto de resultados
Se coloca dentro de una cláusula como **WHERE, HAVING, FROM, o SELECT.**

CONSULTAS ANIDADAS

-- Subconsultas Escalares

```
SELECT nombre  
FROM empleados  
WHERE salario = (SELECT MAX(salario) FROM empleados);
```

-- Subconsultas de Filas Múltiples

```
SELECT nombre  
FROM empleados  
WHERE salario > ANY (SELECT salario FROM empleados WHERE departamento = 'Ventas');
```

-- Subconsultas de Múltiples Columnas

```
SELECT nombre, salario  
FROM (SELECT nombre, salario FROM empleados WHERE departamento = 'Ventas') AS empleados_ventas;
```

-- Subconsultas Correlacionadas

```
SELECT nombre, salario  
FROM empleados e  
WHERE salario > (SELECT AVG(salario) FROM empleados WHERE departamento = e.departamento);
```

CONSULTAS ANIDADAS

-- Subconsulta en la cláusula WHERE

```
SELECT nombre  
FROM empleados  
WHERE salario > (SELECT AVG(salario) FROM empleados);
```

-- Subconsulta en la cláusula FROM

```
SELECT nombre, salario  
FROM (SELECT nombre, salario FROM empleados WHERE salario > (SELECT AVG(salario) FROM  
empleados)) AS empleados_filtrados;
```

-- Subconsulta en la cláusula SELECT

```
SELECT nombre, salario,  
       (SELECT AVG(salario) FROM empleados WHERE departamento = e.departamento) AS  
salario_promedio_departamento  
FROM empleados e;
```

CONSULTAS ANIDADAS

Ventajas

- **Modularidad:** Las subconsultas permiten dividir consultas complejas en partes más manejables.
- **Flexibilidad:** Puedes usar los resultados de la subconsulta en diversas comparaciones o como fuente de datos.
- **Reutilización:** Al reutilizar subconsultas en diferentes cláusulas, puedes evitar la duplicación de código.

Limitaciones

- **Desempeño:** Dependiendo del tamaño de las tablas, las subconsultas, especialmente las correlacionadas, pueden impactar negativamente en el rendimiento.
- **Complejidad:** Aunque ofrecen flexibilidad, las consultas muy anidadas pueden volverse difíciles de leer y depurar.

CONSULTAS ANIDADAS

Operadores de Comparación

Los operadores de comparación permiten comparar un valor de la consulta externa con los resultados de la subconsulta. Los operadores de comparación más comunes son:

= (igual a)

<> (diferente de)

> (mayor que)

< (menor que)

>= (mayor o igual que)

<= (menor o igual que)

```
SELECT nombre  
FROM empleados  
WHERE salario = (SELECT MAX(salario) FROM empleados);
```


CONSULTAS ANIDADAS

Operadores de Conjunto

Los operadores de conjunto permiten comparar un valor con un conjunto de resultados devueltos por una subconsulta. Los operadores más comunes son:

IN

NOT IN

ANY

ALL

EXISTS

NOT EXISTS

CONSULTAS ANIDADAS

-- IN : El operador IN permite verificar si un valor de la consulta externa coincide con algún valor en el conjunto de resultados devuelto por la subconsulta.

```
SELECT nombre  
FROM empleados  
WHERE salario IN (SELECT salario FROM empleados WHERE departamento = 'Ventas');
```

-- NOT IN: El operador NOT IN funciona de manera opuesta a IN y permite verificar si un valor no está presente en el conjunto devuelto por la subconsulta.

```
SELECT nombre  
FROM empleados  
WHERE salario NOT IN (SELECT salario FROM empleados WHERE departamento = 'Ventas');
```

CONSULTAS ANIDADAS

-- ANY : El operador ANY devuelve TRUE si al menos uno de los valores en la subconsulta cumple con la condición de la comparación.

```
SELECT nombre  
FROM empleados  
WHERE salario > ANY (SELECT salario FROM empleados WHERE departamento = 'Ventas');
```

-- ALL : El operador ALL devuelve TRUE si todos los valores devueltos por la subconsulta cumplen con la condición de la comparación.

```
SELECT nombre  
FROM empleados  
WHERE salario > ALL (SELECT salario FROM empleados WHERE departamento = 'Ventas');
```

CONSULTAS ANIDADAS

-- EXISTS : El operador EXISTS se utiliza para verificar si la subconsulta devuelve al menos una fila. Si la subconsulta devuelve una o más filas, EXISTS devuelve TRUE; de lo contrario, devuelve FALSE.

```
SELECT nombre  
FROM empleados e  
WHERE EXISTS (SELECT 1 FROM empleados WHERE departamento = e.departamento HAVING COUNT(*) > 10);
```

-- NOT EXISTS : El operador NOT EXISTS es la versión opuesta de EXISTS. Devuelve TRUE si la subconsulta no devuelve ninguna fila.

```
SELECT nombre  
FROM empleados e  
WHERE NOT EXISTS (SELECT 1 FROM empleados WHERE departamento = e.departamento HAVING COUNT(*) > 10);
```

CONSULTAS ANIDADAS

-- Operadores de Comparación con Subconsultas Correlacionadas

```
SELECT nombre, salario
FROM empleados e
WHERE salario > (SELECT AVG(salario) FROM empleados WHERE departamento = e.departamento);
```

-- BETWEEN

```
SELECT nombre
FROM empleados
WHERE salario BETWEEN (SELECT MIN(salario) FROM empleados WHERE departamento = 'Ventas')
AND (SELECT MAX(salario) FROM empleados WHERE departamento = 'Ventas');
```

-- LIKE

```
SELECT nombre
FROM empleados
WHERE nombre LIKE (SELECT nombre FROM empleados WHERE departamento = 'Ventas');
```

SQL - *Structured Query Language*



Bibliografía:

Introducción a los Sistemas de Bases de Datos - C. J. Date – 7º Edición

- **CAPÍTULO 11: Subconsultas y consultas correlacionadas**
- **CAPÍTULO 12: Operadores de conjunto**
- **CAPÍTULO 17: Seguridad y control de acceso**

Sistemas de Bases de Datos – Conceptos Fundamentales - Elmasri / Navathe – 5º Edición.

- **Capítulo 8: Consultas Anidadas y Correlacionadas**
- **Capítulo 9: Consultas con operadores de conjunto**
- **Capítulo 23: Seguridad en Bases de Datos**

Microsoft Docs – SQL Server:

Consultas anidadas y subconsultas en SQL Server
Administración de roles y permisos en SQL Server
Permisos GRANT, REVOKE y DENY

Próximo encuentro



Clase 8 Joins

Joins: internos y externos (left, right, full). Ejercitación práctica de joins. Consultas complejas y significativas.